

Node.js

Introduction, basic web operations, express.js, database connection

Syllabus

- Node.js - introduction
 - About
 - Installation
 - Features of Node.js
 - Basic modules
- Node.js - low level programming
 - First application - Hello World!
 - Reading query string parameters
 - Files operations
 - Asynchronous vs. Synchronous methods
 - Routing in Node.js
- Express.js - module for easier web application programming
 - Routing
 - Processing Pipeline (middleware)
 - Static files support
 - Reading data from a form (or from query string)
 - Basic template files with placeholder
 - Database connection (SELECT, UPDATE)
 - URL parameters
 - Error handling

Introduction 1/4

- Node.js is an open-source server-side environment.
 - Node.js is free.
- Node.js runs on various platforms (Windows, Linux, Unix, Mac OS X, etc.).
- Node.js uses JavaScript on the server.
 - Before Node.js, JavaScript was primarily used for client-side scripting in web browsers.
- Node.js uses asynchronous programming!
- Node.js is not a framework or a programming language.
- Node.js is currently the most popular and widely used open-source server-side solution for building small, large, and any-size web applications.

Introduction 2/4

- Node.js handles requests asynchronously by **design**.
- It sends the request to the application's handler.
- It is immediately ready to handle the next request.
- Once the handler prepares the response, the server returns the content to the client.
 - Node.js eliminates the wait and simply continues with the next request.
 - Node.js runs single-threaded, non-blocking, asynchronous programs. This approach is very memory efficient.

Introduction 3/4

What can *Node.js* do? What any application can do, and in the context of web applications:

- Node.js can generate dynamic page content,
- Node.js can create, open, read, write, delete, and close files on the server,
- Node.js can collect form data,
- Node.js can add, delete, and modify data in your database.

What is a *Node.js* file?

- *Node.js* files contain tasks that will be executed on specific events.
- A typical event is an attempt to access a port on the server.
- *Node.js* files must be initialized on the server before they can have any effect.
- *Node.js* files have the ".js" extension.

Introduction 4/4

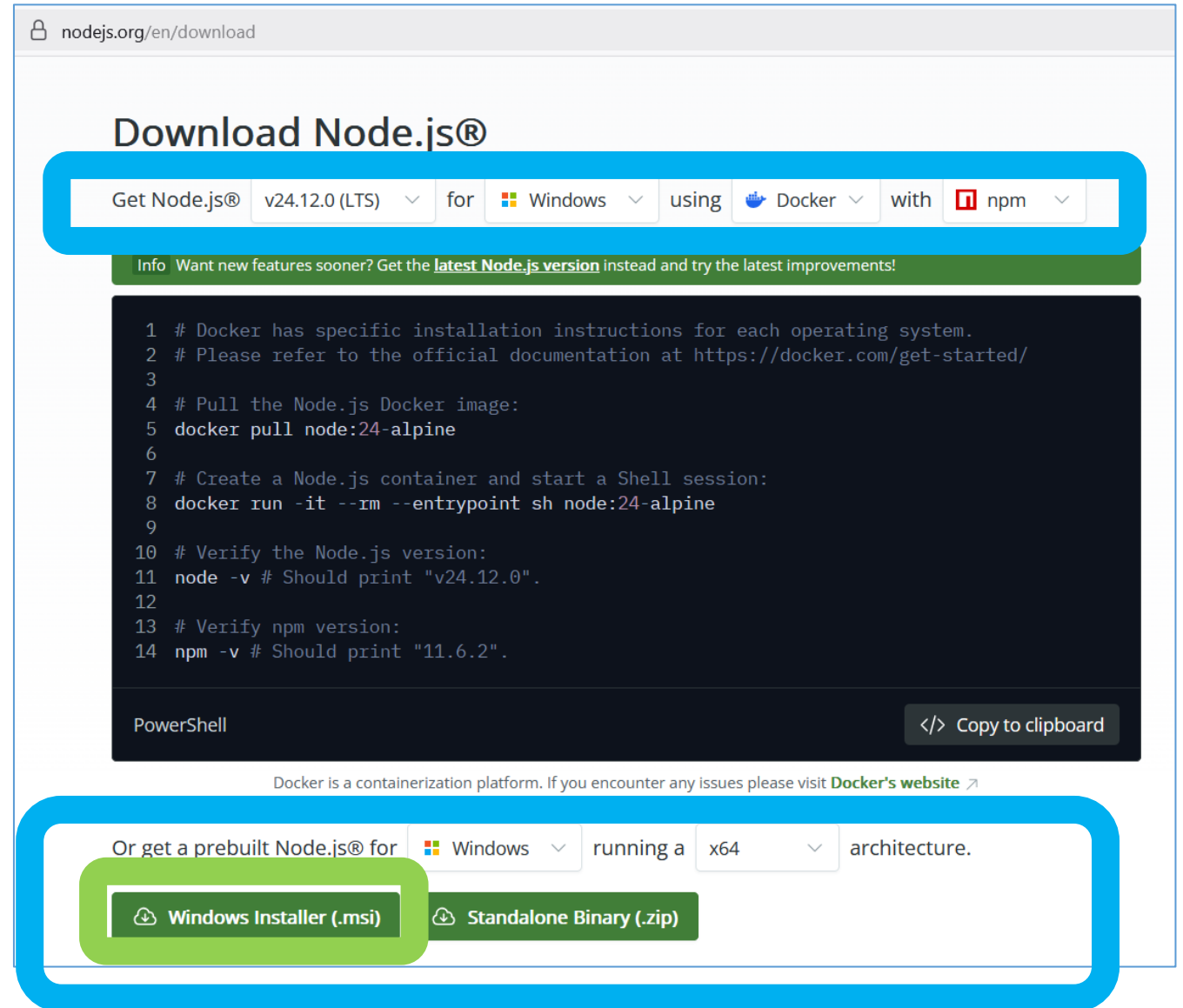
- Node.js is a JavaScript runtime environment based on the Chrome V8 JavaScript engine.
- It operates **outside the browser**/browser environment.
- It interacts directly with the operating system, providing applications with access to the operating system APIs, including access to the file system, system libraries, and running processes, including HTTP servers.
- Node.js's unique feature is its use of the JavaScript event loop. This allows you to create non-blocking I/O applications that easily handle multiple simultaneous events.

Installation 1/4

- **Administrator privileges** in the operating system for installation.
- There are essentially two different versions of Node:
 - a) **LTS** (Long Term Support), which is the most stable and tested version, and
 - b) **Current**, which includes new features still in development.
- It is **recommended** to install the **LTS version**. This version usually contains fewer bugs and is suitable for production use.
- The default options during installation are suitable for most developers.

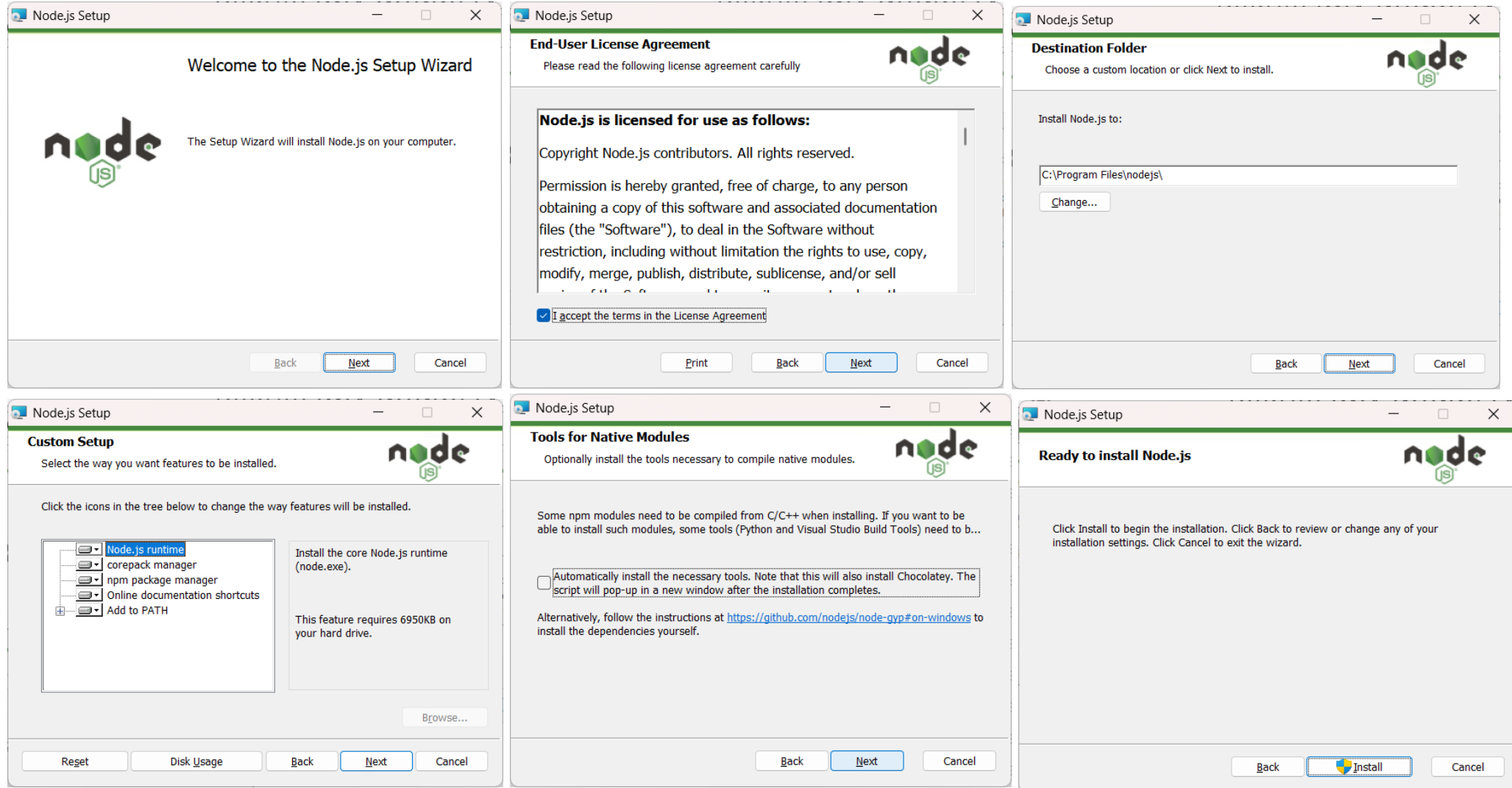
Installation 2/4

- Node.js official website:
`https://nodejs.org`
 - This website has installation instructions for Node.js
- For download start from url:
`nodejs.org/en/download`
- Choose a version for your operating system
 - right screenshot for Windows operating system.

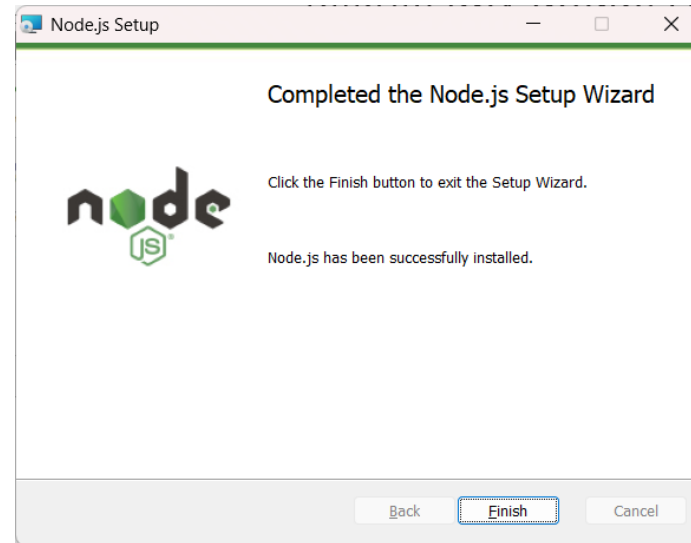
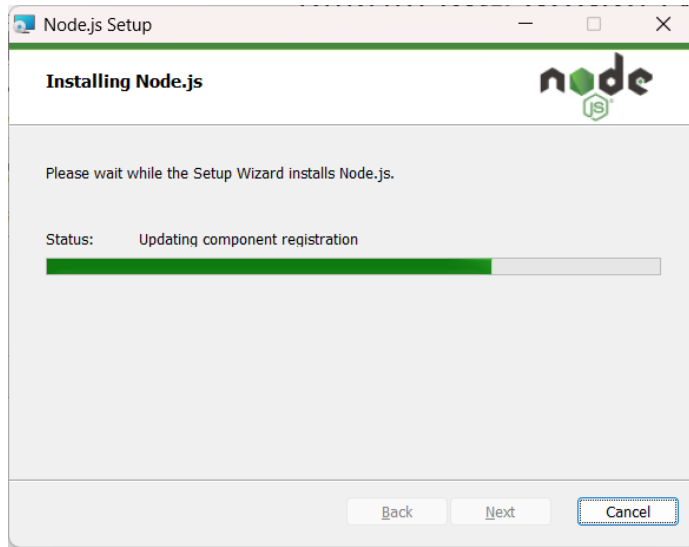


Installation 3/4

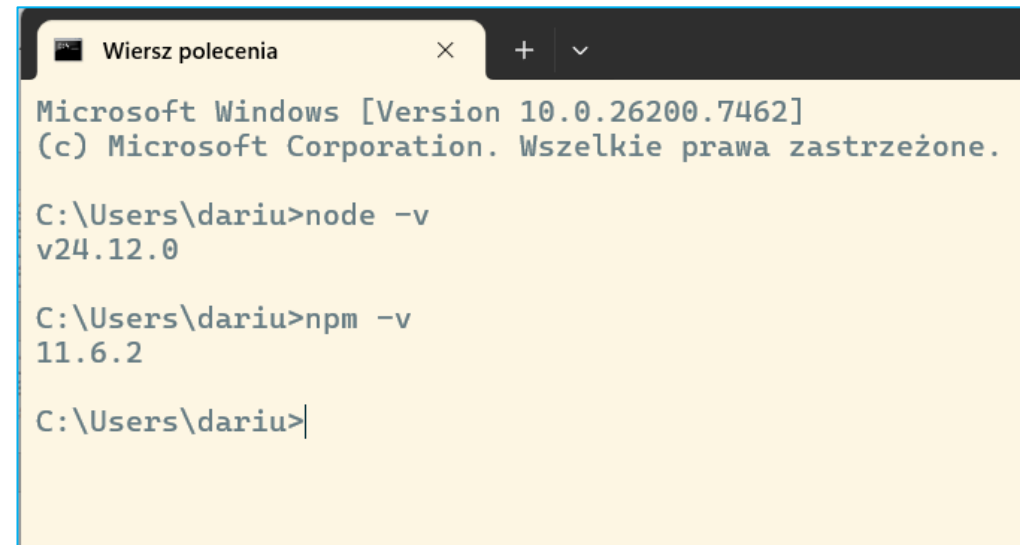
- Follow these screenshots



Installation 4/4



- open CMD and write 'node -v'.
- you have to see output line with the version of Node.js.
- you can also check the version of npm

The image shows a Windows Command Prompt window titled 'Wiersz polecenia'. It displays the output of the 'node -v' and 'npm -v' commands. The text is as follows:

```
Microsoft Windows [Version 10.0.26200.7462]  
(c) Microsoft Corporation. Wszelkie prawa zastrzeżone.  
  
C:\Users\dariu>node -v  
v24.12.0  
  
C:\Users\dariu>npm -v  
11.6.2  
  
C:\Users\dariu>
```

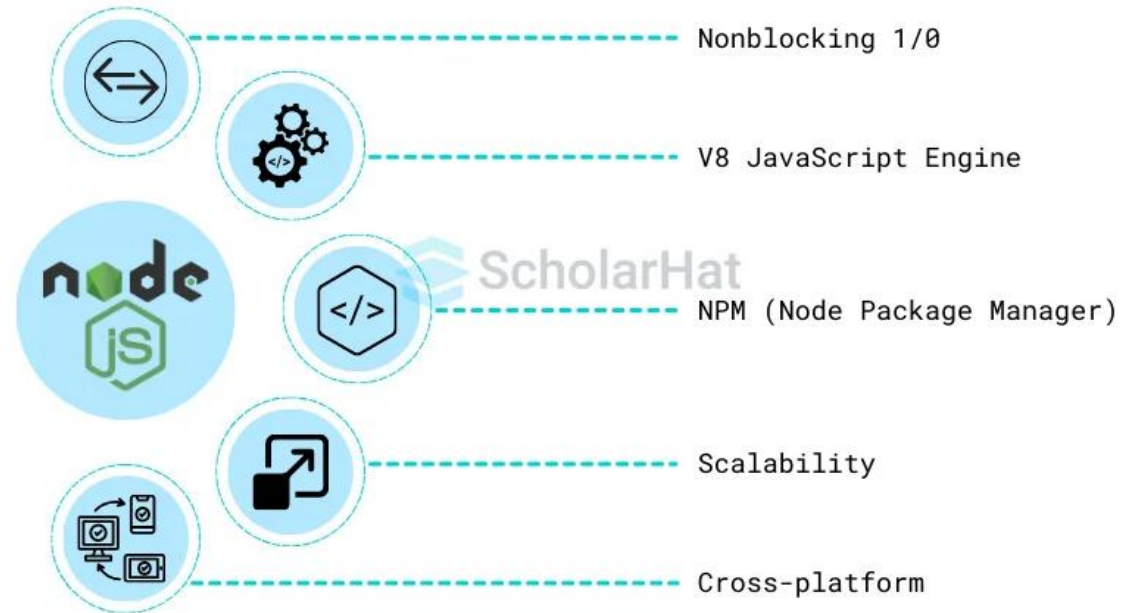
Advantages of *Node.js*

- High performance.
- Application code – "good old JavaScript" is used here, meaning less time is spent on "context switching" when working with different programming languages on the client and server sides.
- JavaScript in this application is a relatively new programming language. It offers many improvements compared to other languages commonly used on the server side (e.g., PHP, Python, etc.). Additionally, many new and popular languages are compiled/translated to JavaScript, so you can use CoffeeScript, ClojureScript, TypeScript, Scala.js, LiveScript, and others.
- The Node Package Manager (NPM) provides thousands of packages for use. It has a best-in-class dependency resolution mechanism and can be used to automate application build tools.
- Node.js is portable and available for various operating systems: Microsoft Windows, Linux, macOS, Solaris, FreeBSD, OpenBSD, WebOS, and NonStop OS. It is also supported by network service providers who provide the appropriate infrastructure and documentation for hosting Node applications,

Key Features of Node JS

- Non-blocking I/O: The non-blocking nature of I/O allows applications to handle multiple requests simultaneously, resulting in faster response times.
- JavaScript Engine V8: Google's V8 engine, known for its speed and efficiency, quickly executes JavaScript code.
- NPM (Node Package Manager): NPM simplifies dependency management by giving developers access to a large ecosystem of packages and libraries for their applications.
- Scalability: It supports seamless vertical and horizontal scalability, making it suitable for projects of any size.
- Cross-platform: It is compatible with various operating systems, allowing for easy deployment and adaptation.

Distinct Feature of NodeJS



Źródło: <https://www.scholarhat.com/tutorial/nodejs/nodejs-and-expressjs>

Node.js - Application area source:

- Node.js can be used to create following types of applications.
 - 1.E-Commerce Web Applications
 - 2.Social Media Applications
 - 3.Proxy Server
 - 4.Real-time Services
 - 5.Real-time data Applications like Multiplayer Games, Stock Trading, Chat App etc.
 - 6.Data Streaming Applications
 - 7.Network Applications
 - 8.High Concurrency Applications
 - 9.File Uploading Tools
 - 10.Process Monitoring Tools
 - 11.HTTP Web Server
- source: <https://www.scholarhat.com/tutorial/nodejs/what-is-nodejs-and-why-to-use-it>

Built-in *Node.js* modules

- *Node.js* has a set of built-in modules that can be used without any additional installation.
- Here's a list of built-in modules for *Node.js* version 6.10.3:

Module	Description
assert	Provides a set of assertion tests
buffer	To handle binary data
child_process	To run a child process
cluster	To split a single Node process into multiple processes
crypto	To handle OpenSSL cryptographic functions
dgram	Provides implementation of UDP datagram sockets
dns	To do DNS lookups and name resolution functions
domain	Deprecated. To handle unhandled errors
events	To handle events
fs	To handle the file system
http	To make Node.js act as an HTTP server
https	To make Node.js act as an HTTPS server.
net	To create servers and clients
os	Provides information about the operation system
path	To handle file paths
punycode	Deprecated. A character encoding scheme
querystring	To handle URL query strings
readline	To handle readable streams one line at the time
stream	To handle streaming data
string_decoder	To decode buffer objects into strings
timers	To execute a function after a given number of milliseconds
tls	To implement TLS and SSL protocols
tty	Provides classes used by a text terminal
url	To parse URL strings
util	To access utility functions
v8	To access information about V8 (the JavaScript engine)
vm	To compile JavaScript code in a virtual machine
zlib	To compress or decompress files

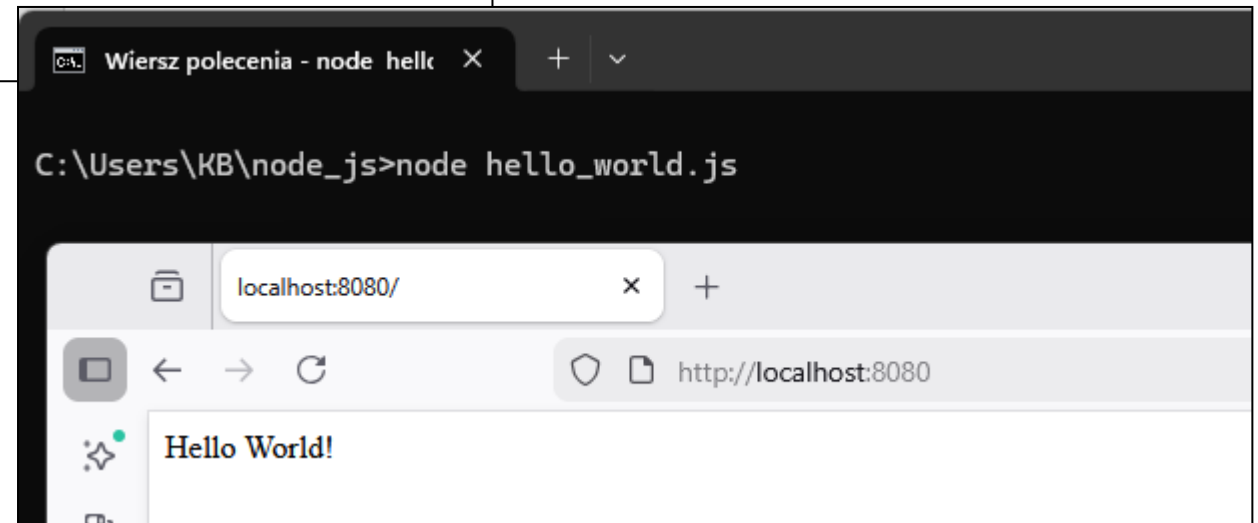
First program - "Hello World"

- The file `hello_world.js` with below code.
- The code tells the computer to write "Hello World!" if anyone (e.g., a web browser) tries to access the computer through port 8080.
- Running in the console: `node hello_world.js`

```
// file: hello_world.js
// source: https://www.w3schools.com/nodejs/

var http = require('http');

http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.end('Hello World!');
}).listen(8080);
```



Modules

- Modules are similar to JavaScript libraries—they are sets of functions that you want to include in your application.
- Built-in modules - *Node.js* has a set of built-in modules that you can use without any additional installation.
- Including modules - to include a module, use the module-named function: `require()`
 - `var http = require('http');`
 - now your application has access to the HTTP module and can create a server
- You can create your own modules and easily incorporate them into your applications (this is not the scope of this lecture).

Elements of the first program

- Adding the HTTP module:
 - `var http = require('http');`
- Create an HTTP Server object using the `createServer()` method, which requires a two-argument function as an argument.
 - The above-mentioned function receives an object containing the request data as its first argument and a second argument, which is the response object (to that request). This second object should be populated with data using the appropriate methods of this object.
- The resulting server should then be launched on the selected port using the `listen()` method.

```
// file: hello world.js
// source: https://www.w3schools.com/nodejs/

var http = require('http');

http.createServer(function (req, res) {
    res.writeHead(200, {'Content-Type': 'text/html'});
    res.end('Hello World!');
}).listen(8080);
```

Request handling function

- In this simple program, for each request, the request handler function `function (req, res)`
 - fills the header with data using a call to the `writeHead()` method on the `res` object.
 - The first argument is the response code.
 - The second is an object of header pairs: the header name and the header value, joined by a colon ':'. In this program, this simply tells the browser to treat the response as an HTML page.
 - The response body is filled in using successive `write()` methods and ending with the `end()` method. In this case, since the body is only a single line of text, this can only be done with the `end()` method, but it can also be written as:
 - `res.write('Hello World!');` //write a response to the client
 - `res.end();` //end the response

```
// file: hello world.js
// source: https://www.w3schools.com/nodejs/

var http = require('http');

http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.end('Hello World!');
}).listen(8080);
```

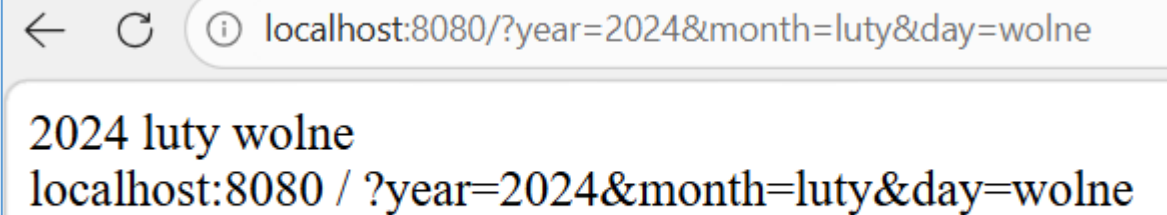
Reading query string parameters

- The function passed to `http.createServer()` has a `req` argument that represents the request from the client in the form of an object (the `http.IncomingMessage` object – i.e. the incoming message).
- This object has a property called `"url"` that contains the part of the URL following the domain name.
- There's a `URL` class. It's globally available, so you don't even need to `require('url')` to use this class. The code for the following URL is on the next slide:
 - <http://localhost:8080/?year=2024&month=luty&day=wolne>
- This class also makes it easier to access other url parameters from the request (collection `searchParams`, properties like `host`, `pathname`, `search` etc.).
- After creating variable `myUrl` from another elements of `req` by below code:

```
const baseUrl = 'http://' + req.headers.host;  
const myUrl = new URL(req.url, baseUrl);
```
- you can access query parameters by `myUrl.searchParams.get(nameOfParam)`

Code for accessing query parameters

```
var http = require('http');
// file: year_month.js
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html; charset=utf-8'});
  // In the new standard, the URL requires the full address (including the host)
  // req.url is just a path (e.g., /?year=2026), so we add the database
  const baseURL = 'http://' + req.headers.host;
  const myUrl = new URL(req.url, baseURL);
  // We get query parameters
  const q = myUrl.searchParams;
  const year = q.get('year') || "";
  const month = q.get('month') || "";
  const day = q.get('day') || "";
  let txt = `${year} ${month} ${day}<br>`;
  // We get the properties from the myUrl object, not from req
  txt += myUrl.host + " " + myUrl.pathname + " " + myUrl.search;
  res.end(txt);
}).listen(8080);
```



← ↻ ⓘ localhost:8080/?year=2024&month=luty&day=wolne

2024 luty wolne
localhost:8080 / ?year=2024&month=luty&day=wolne

File operations (mainly text files)

- To include the filesystem module, use the following require() method:

```
var fs = require('fs');
```

- Another way to access files:

```
import fs from 'fs';  
// import { readFile, writeFile } from 'fs/promises';
```

- Common uses for the filesystem module:

- Reading files
- Creating files
- Updating files
- Deleting files
- Renaming files
- Folder operations

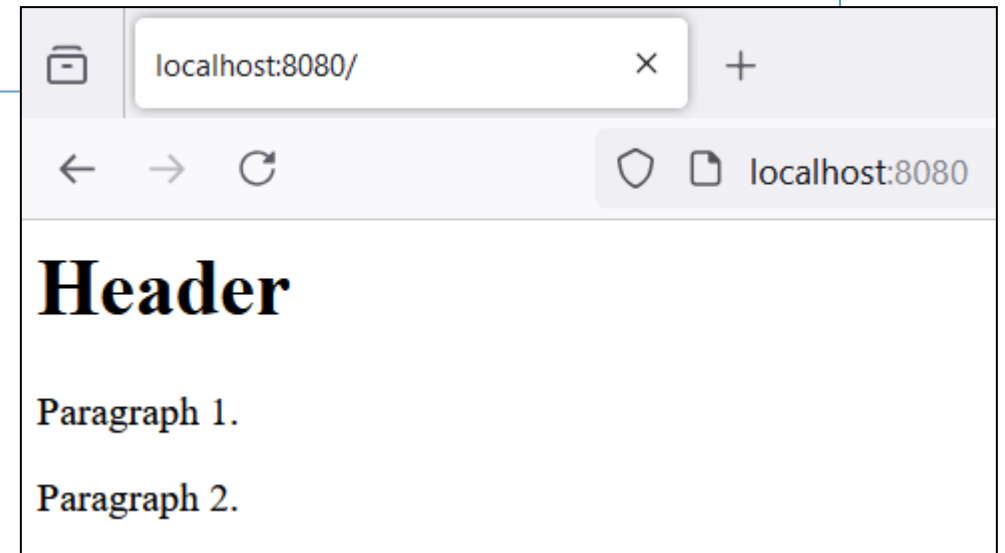
- Performance tip: For large files, consider using streams (`fs.createReadStream` and `fs.createWriteStream`) which read/write in batches to avoid high memory usage.
- Note: Always handle errors during file operations to prevent your application from crashing.
- Best practice: Always specify a character encoding (e.g., "utf8") when reading text files to get a string of characters instead of a string of bytes.

Reading a file

- Reading a file is done by using the `fs.readFile()` function, which includes the file path, encoding type, and a method to be executed **when the file read is complete** (or an error occurs).
- Therefore, the file reading operation is asynchronous.

```
var http = require('http');
var fs = require('fs');
// file: demo_file.js
// source: https://www.w3schools.com/nodejs/
http.createServer(function (req, res) {
  //Open a file on the server and return its content:
  fs.readFile('./demo_file.html', 'utf8', (err, data) => {
    if (err) {
      console.error('Error reading file:', err);
      return;
    }
    console.log('File content:', data);
    res.writeHead(200, {'Content-Type': 'text/html'});
    res.write(data);
    return res.end();
  });
}).listen(8080);
```

```
<!DOCTYPE html>
<html>
<body>
  <!-- file: demo_file.html
       source: https://www.w3schools.com/nodejs/-->
  <h1>Header</h1>
  <p>Paragraph 1.</p>
  <p>Paragraph 2.</p>
</body>
</html>
```



Asynchronous vs. Synchronous Readings

- **Synchronous** - `readFileSync()` method (`hello_world2.js` example) without a third parameter, but throwing an exception.
 - Thread execution is blocked until the read is complete.
 - May cause server-wide delays.
 - Easy to understand.
- **Asynchronous** - `readFile()` method (`hello_world3.js` example)
 - Non-blocking execution.
 - Better performance.
 - More complex to maintain.
 - Uses callbacks, promises, or `async/await`.

Expected text layout from files and code

- The examples on the next slide attempt to assemble a response consisting of the first file (`first.html`), then appending a string of JavaScript code, and finally appending a string from the second file (`last.html`).
- So the order have to be:
 - a text from `first.html` file.
 - a text generated by code in JavaScript (in `.js` file)
 - a text from `last.html` file.

```
<!DOCTYPE html>
<!-- first.html -->
<html>
  <head>
    <title>*** Hello World 2025!</title>
  </head>
<body>
```

text from Javascript code

```
<!-- last.html -->
</body>
</html>
```


Synchronous version

- Now we need to handle exceptions in try/catch blocks.

```
var http = require('http');
var fs = require('fs');
// file: hello_world2.js
var text_html="";
```

```
try {
  // Read file synchronously
  const first = fs.readFileSync('first.html', 'utf8');
  text_html=text_html+first;
} catch (err) {
  console.error('Error reading file:', err);
}
```

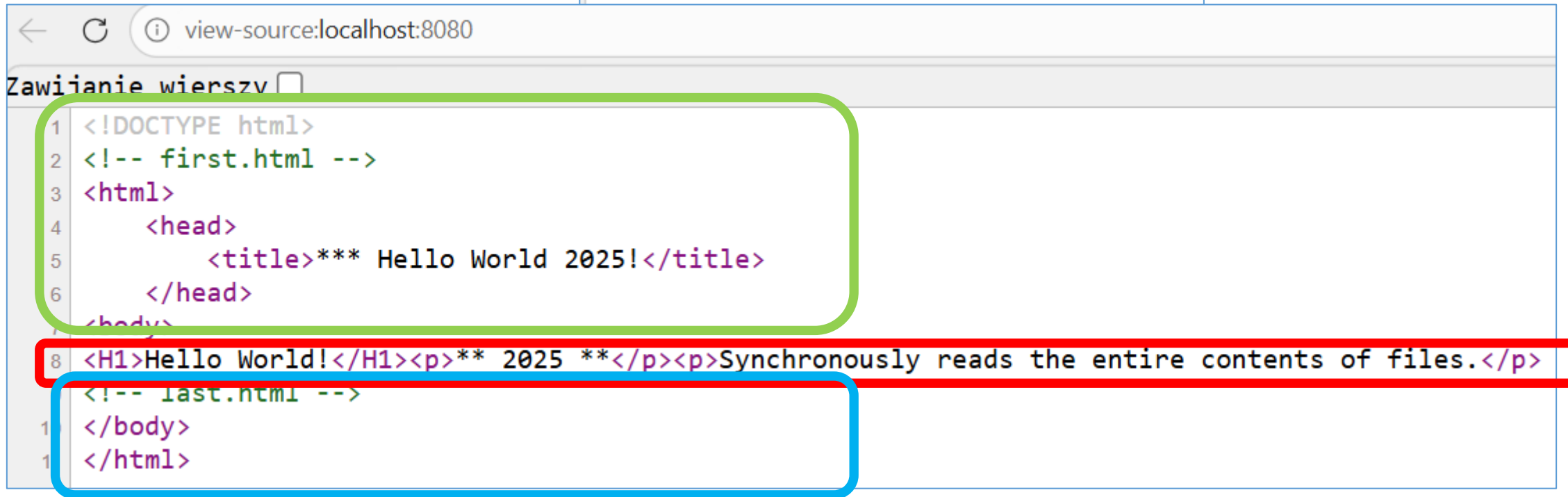
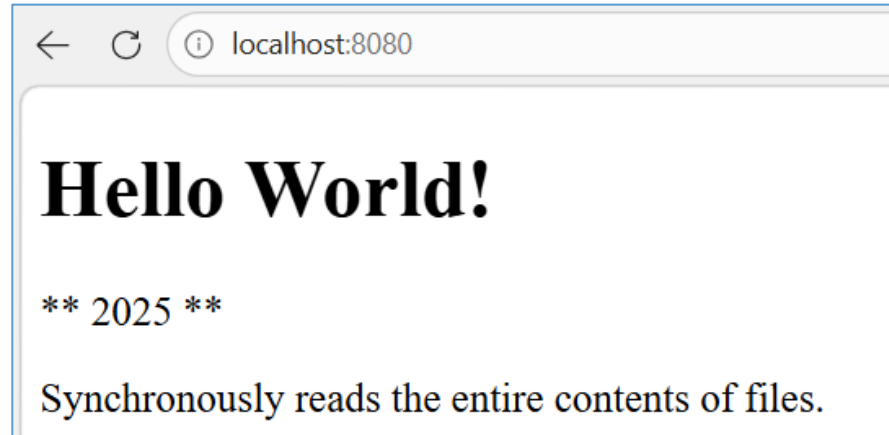
```
text_html=text_html+'<H1>Hello World!</H1>';
text_html=text_html+'<p>** 2025 **</p>';
text_html=text_html+'<p>Synchronously reads the entire contents of files.</p>';
```

```
try {
  // Read file synchronously
  const last = fs.readFileSync('last.html', 'utf8');
  text_html=text_html+last;
} catch (err) {
  console.error('Error reading file:', err);
}
```

```
http.createServer(function (req, res) {
  //Open a file on the server and return its content:
  // console.log('File content all:', text_html);
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.write(text_html);
  res.end()
}).listen(8080);
```

The effect of the synchronous version

- Both the view in the browser and the generated HTML code are correct.
- The order of the text fragment is preserve.



A screenshot of a web browser window showing the source code of the page. The address bar shows 'view-source:localhost:8080'. The page title is 'Zawianie wierszv'. The source code is as follows:

```
1 <!DOCTYPE html>
2 <!-- first.html -->
3 <html>
4   <head>
5     <title>*** Hello World 2025!</title>
6   </head>
7   <body>
8     <H1>Hello World!</H1><p>** 2025 **</p><p>Synchronously reads the entire contents of files.</p>
9   <!-- last.html -->
10  </body>
11  </html>
```

The source code is highlighted with three colored boxes: a green box around lines 1-6, a red box around line 8, and a blue box around lines 9-11.

The asynchronous version

- This is a modification of the synchronous version, but it is **incorrectly** executed because it abstracts from asynchronous execution (it **does not use synchronization** mechanisms between threads).

```
var http = require('http');
var fs = require('fs');
// file: hello_world3.js
var text_html="";

fs.readFile('first.html', 'utf8', (err, first) => {
  if (err) {
    console.error('Error reading file:', err);
    return;
  }
  text_html=text_html+first;
  return ;
});

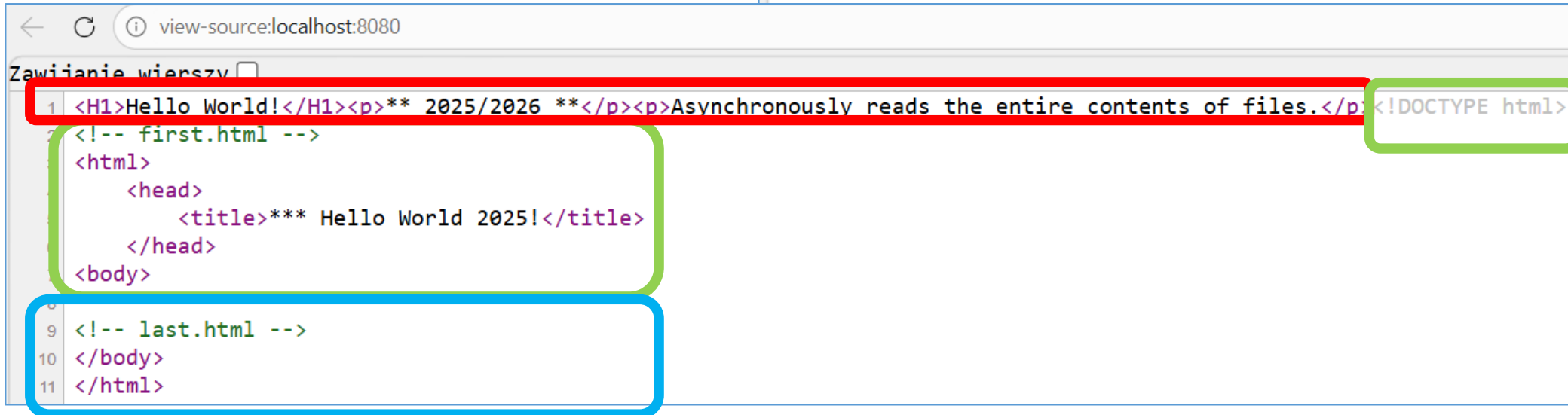
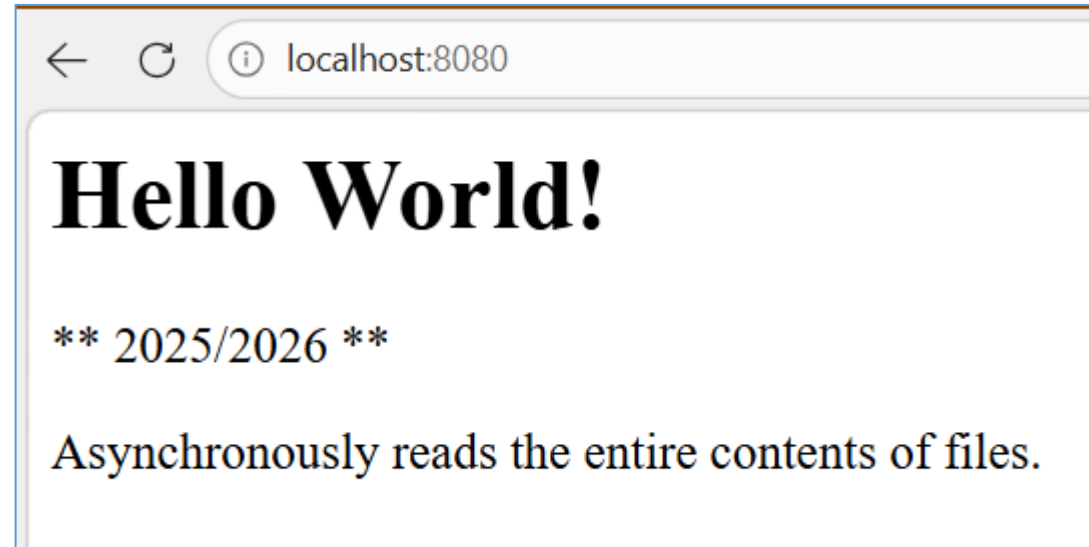
text_html=text_html+'<H1>Hello World!</H1>';
text_html=text_html+'<p>** 2025/2026 **</p>';
text_html=text_html+'<p>Asynchronously reads the entire contents of files.</p>';

fs.readFile('last.html', 'utf8', (err, last) => {
  if (err) {
    console.error('Error reading file:', err);
    return;
  }
  text_html=text_html+last;
  return ;
});

http.createServer(function (req, res) {
  //Open a file on the server and return its content:
  // console.log('File content all:', text_html);
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.write(text_html);
  res.end()
}).listen(8080);
```

The effect of the asynchronous version

- Although the view in the browser appears correct, in reality the generated HTML code is incorrect.
- First, the text from the JavaScript code was added, and only then, asynchronously, the texts from the two file reading methods were added.



Other file operations from the fs module

Creating a file	
<code>fs.appendFile('mynewfile1.txt', 'Hello content!', function (err) {...});</code>	The <code>fs.appendFile()</code> method adds the specified content to a file. If the file does not exist, it will be created:
<code>fs.open('mynewfile2.txt', 'w', function (err, file) {...});</code>	The <code>fs.open()</code> method takes a "flag" as the second argument. If the flag is "w" for "writing", the specified file is opened for writing. If the file does not exist, an empty file is created:
<code>fs.writeFile('mynewfile3.txt', 'Hello content!', function (err) {...});</code>	The <code>fs.writeFile()</code> method overwrites the specified file and contents, if they exist. If the file does not exist, a new file is created with the specified contents:
Updating file	
<code>fs.appendFile('mynewfile1.txt', 'This is my text.', function (err) {...});</code>	The <code>fs.appendFile()</code> method appends the specified content to the end of the specified file:
<code>fs.writeFile('mynewfile3.txt', 'This is my text', function (err) {...});</code>	The <code>fs.writeFile()</code> method overwrites the specified file and its contents.
Others	
<code>fs.rename('mynewfile1.txt', 'myrenamedfile.txt', function (err) {...});</code>	The <code>fs.rename()</code> method renames a file.
<code>fs.unlink('mynewfile2.txt', function (err) {...});</code>	The <code>fs.unlink()</code> method removes the file.

Routing in Node.js

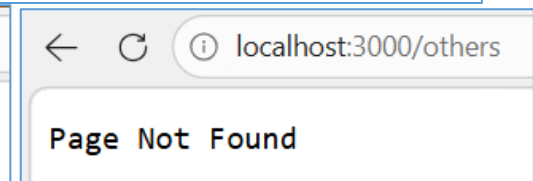
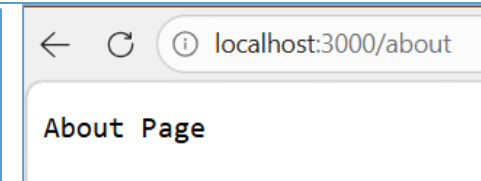
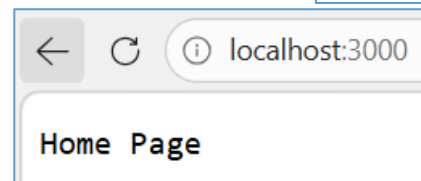
- Routing describes how a server responds to various HTTP requests. *Node.js* allows routing to be managed directly within the server code.
- *Node.js* does **not have built-in** routing features.
- Instead, developers must manually evaluate the URL and method of each incoming request to determine how to properly handle and respond to them.
- This means that routing logic must be implemented in the application code, typically using conditional statements or frameworks like Express.js, which provide more structured routing capabilities.
- By examining the URL and method of each request, developers can dynamically modify responses to meet individual application requirements.

Routing in Node.js - an example

- This *Node.js* script creates a basic HTTP server capable of handling multiple routes (e.g., / for the home page and /about for the About page).
- If a user navigates to an unknown route, the server will display a 404 error with the message "Page not found."
- This demonstrates the basic routing logic for a Node.js server.
 - The example is not entirely correct because it doesn't check which HTTP method (GET/POST/...) was used in the request.
- <http://localhost:3000/>
<http://localhost:3000/about>
<http://localhost:3000/unknown>

```
const http = require('http');  
// file: routing.js  
// source: https://www.scholarhat.com/tutorial/nodejs/nodejs-and-expressjs  
const server = http.createServer((req, res) => {  
  if (req.url === '/') {  
    res.statusCode = 200;  
    res.end('Home Page');  
  } else if (req.url === '/about') {  
    res.statusCode = 200;  
    res.end('About Page');  
  } else {  
    res.statusCode = 404;  
    res.end('Page Not Found');  
  }  
});  
  
server.listen(3000, () => {  
  console.log('Server running at http://localhost:3000/');  
});
```

- **This solution is not very convenient.**
 - let's use the `express.js` module.



Module `express.js`

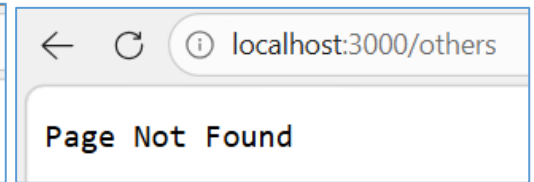
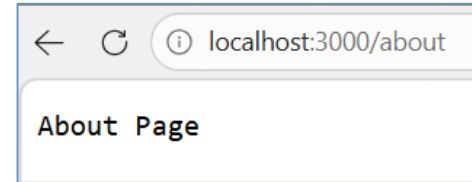
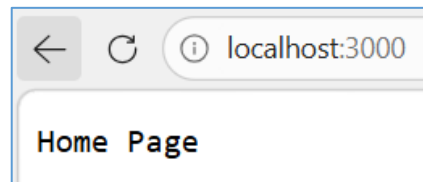
- Routing
- Processing Pipeline (middleware)
- Static files support
- Reading data from a form (or from query string)

Module express.js - routing

- To **simplify routing** and **other web server programming**, it's worth using the `express.js` module.
- Install from the **command line (as admin)** in the project folder (this folder shouldn't be on a cloud drive):
 - `npm install express`
- The installation is local, so it must be performed **separately for each project** – the recommended method.
- The previous code can now be written much cleaner.
- The app uses inside the `http` object.

```
const express = require('express'); // We import Express
const app = express();               // We create an application instance
const port = 3000;                   // We set the port number
// file: ex_routing.js

app.get('/', (req, res) => {
  res.send('Home Page');
});
app.get('/about', (req, res) => {
  res.send('About Page');
});
app.use((req, res) => {
  res.status(404).send('Page Not Found');
});
app.listen(port, () => {
  console.log('Server running at http://localhost:3000/');
});
```



Express.js Processing Pipeline

- Successive executions of the `get()`, `post()`, `use()`, and other methods create a request processing pipeline.
- The `use()` method **can have** a **first** parameter that is a path prefix. If it doesn't, it defaults to '/'.
- Other methods must have a first parameter that is an **exact path**, but **text patterns, parameters, or regular expressions** can be used.
- The **next parameter** for all methods is a **function** with parameters (`req`, `res`), which, however, compared to pure Node.js behavior, may have **new properties** added by previous methods in the processing pipeline.
- If the request should proceed to the next element in the pipeline (even though it matches the path), the `next()` method should be called at the end.

Reading data from a form

- File: `form_ex.js`
- The application should return the prepared page with the form (saved in the `form_ex.html` file) at the request `GET /form.html`.
- After clicking "Submit", the data is to be sent to the request handler as a POST method to the address `/show.html`.
- The data will then be received on the server and displayed in a table using the partially prepared page in the `form_table.html` file.
 - This file contains a marker – the text `"{mydata}"`, which is where the JavaScript-generated text will be inserted.

Expected action

Example form:

First name:
Anna

Last name:
Nowak

Index number
00000

Is active?
☒ Yes
☐ No
Submit

Example form:

First name:
Anna

Last name:
Nowak

Index number
12345

Is active?
☐ Yes
☒ No
Submit

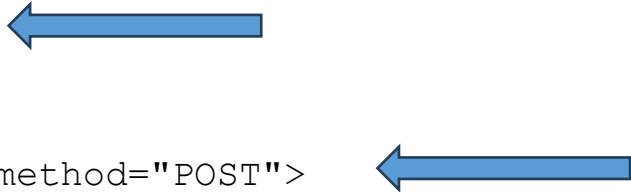
Data from form as a table

First name	Anna
Last name	Nowak
Index number	12345
Is active	0

Form file - form_ex.html

- We want to use style sheets
 - in the `public/css/style.css` file
- As intended: POST method to `/show.html`.

```
<!DOCTYPE html>
<!--File: form_ex.html-->
<html><head>
<link rel="stylesheet" href="/css/style.css">
</head><body>
<h1> Example form: </h1>
<form action="http://localhost:3000/show.html" method="POST">
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname" placeholder="Anna" required><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname" placeholder="Nowak" required><br><br>
  <label for="nr_index">Index number</label><br>
  <input type="number" id="index_no" name="index_no" placeholder="00000" required><br><br>
  <label for="is_active">Is active?</label><br><br>
  <input type="radio" id="is_active" name="is_active" value="1" checked>Yes<br>
  <input type="radio" id="is_active" name="is_active" value="0">No<br>
  <input type="submit" value="Submit">
</form>
</body></html>
```



File form_ex.js - 1/2

- The `express.js` module can "extract" data from the request body sent via POST in its default format into properties that will be in `req.body`. If this is needed, add the appropriate `app.use()` somewhere at the beginning of the processing pipeline:
 - `app.use(express.urlencoded({ extended: true }));`
- Similarly, if we want to handle all requests for static files that are stored in the `/public` folder (and subfolders) by default:
 - `app.use(express.static('public'));`
- Handling a GET `/form.html` request is simply sending back the contents of the `form_ex.html` file.

```
const express = require('express');
const fs = require('fs');
const app = express();
const port = 3000;
// file: form_ex.js

// Form parser (Content-Type: application/x-www-form-urlencoded)
app.use(express.urlencoded({ extended: true }));

// Everything from the public folder will be available
// immediately in this application: css/style.css
app.use(express.static('public'));

app.get('/form.html', (req, res) => {
  res.send(fs.readFileSync('form_ex.html', 'utf8'));
});
```

File form_table.html

- This file contains most of the elements of a valid HTML file, but at a specific location, there's a `{mydata}` tag indicating where text should be replaced.
- This file also uses external styles.

```
<!DOCTYPE html>
<!--File: form_table.html-->
<html>
  <head>
    <link rel="stylesheet" href="/css/style.css">
  </head>
  <body>
    <h1> Data from form as a table </h1>
    <table>
      <tbody>
        {mydata}
      </tbody>
    </table>
  </body>
</html>
```



File form_ex.js - 2/2

- The `generateTableBody()` method prepares HTML text for table rows with data retrieved from `req.body`.
- Handling a POST `/show.html` involves loading the `form_table.html` file, generating the table content, and inserting the result in place of `{mydata}`.

```
let generateTableBody=function(req){
  let data=req.body; // data from body of request
  // let data=req.query; // data from query string of request
  let text=`<tr><td>First name</td><td>${data.fname}</td></tr>`;
  text+=`<tr><td>Last name</td><td>${data.lname}</td></tr>`;
  text+=`<tr><td>Index number</td><td>${data.index_no}</td></tr>`;
  text+=`<tr><td>Is active</td><td>${data.is_active}</td></tr>`;
  return text;
}

app.post('/show.html', (req, res) => {
  //app.get('/show.html', (req, res) => {
    let text=fs.readFileSync('form_table.html', 'utf8')
    let asTable=generateTableBody(req);
    text=text.replace("{mydata}",asTable);
    res.send(text);
  });

  app.use((req, res) => {
    res.status(404).send('Page Not Found');
  });

  app.listen(port, () => {
    console.log('Server running at http://localhost:3000/');
  });
});
```


Changes to the GET request

- The program works correctly.
- If the form data were to be sent using the GET method, only three changes would be necessary:
 - In the `form_ex.html` file, change the beginning line of the form to:
 - `<form action="http://localhost:3000/show.html" method="GET">`
 - In the `form_ex.js` file, change `app.post()` to `app.get()`:
 - `app.get('/show.html', (req, res) => {`
 - In the same file, change the data source from `req.body` to `req.query`, as the "extraction" will now be performed from the query string and inserted as properties into `req.query`:
 - `let data=req.query; // data from query string of request`
- At this point, the form data will be included in the URL.
- Of course, this allows you to send some data in the request body and some in the query string in a single request.

Action after change on GET request

The diagram illustrates the process of handling a GET request after a form submission. It consists of three browser window screenshots:

- Initial Form:** A browser window at `http://localhost:3000/form.html` showing a form with fields for First name (Anna), Last name (Nowak), Index number (00000), and Is active? (Yes selected). A blue box highlights the address bar.
- Form Submission:** The same browser window after submission, with the Submit button highlighted by a green box. The form fields now show First name (Anna), Last name (Nowak), Index number (12345), and Is active? (No selected).
- Data Table:** A browser window at `http://localhost:3000/show.html?fname=Anna&lname=Nowak&index_no=12345&is_active=0` displaying the data from the form as a table. A blue box highlights the address bar, and a line connects the Submit button from the previous window to this one.

Data from form as a table

First name	Anna
Last name	Nowak
Index number	12345
Is active	0

Database connection

and:

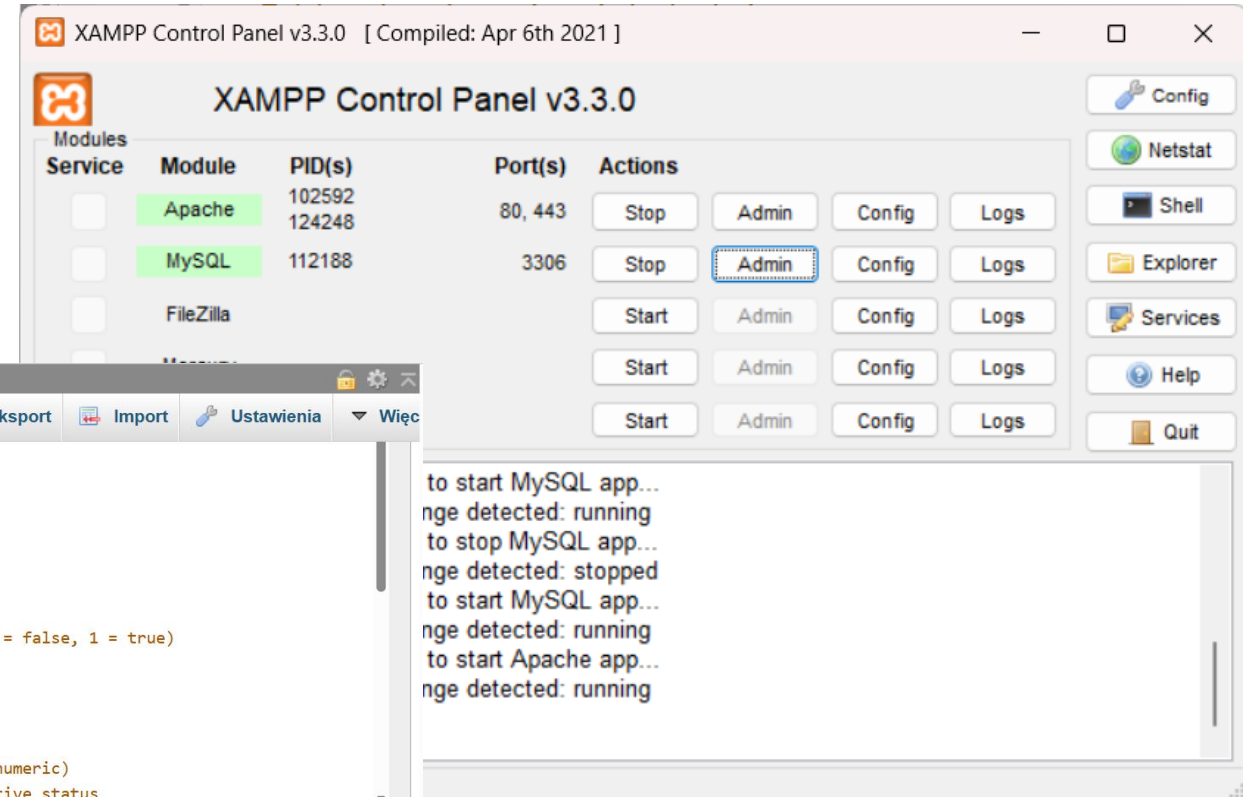
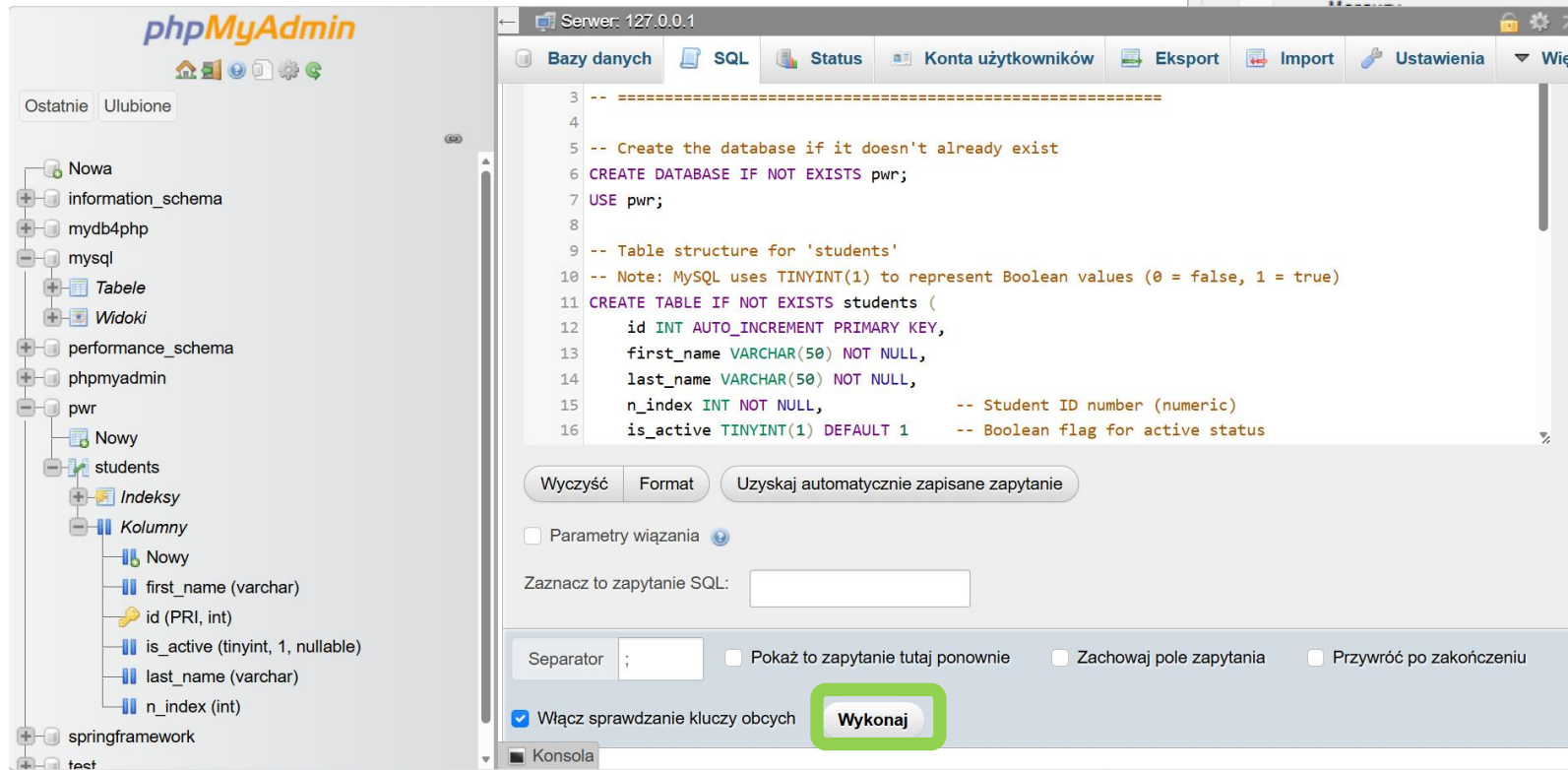
- Basic template files with placeholder
- URL parameters
- Error handling

Application with database connection

- Application assumptions:
 - A student database created outside the application with one simple table: students.
 - A `pwr` database in MySQL within XAMPP, with a `db_creation.sql` file.
 - A user with access to this database created outside the application:
 - user `admin_pwr`, password `admin12345`, `db_create_user_pwr_admin.sql` file.
 - A tabular website with database data and a button for creating new data. In each row, a button to display the selected student's data (via ID) on a separate page.
 - url path: `/students` - template file: `student_index.html`
 - Webpage with the selected student's data
 - url path: `/students/view/:id` - template file: `student_view.html`
 - Webpage with a form for entering new student data
 - url path: `/students/add` - template file: `student_add.html`
 - Sending data from the form to the server via POST
 - url path `/students/add`
 - After reading the data from the POST method, the new student is added to the database and the user is redirected to the first page with the table.
 - redirection to path `/students`

Creating a database and its user pwr_admin

- The step depends on the database you're using.
- Below is a presentation in phpMyAdmin.



Creating and closing a database connection 1/2

- Installing the *mysql2* module
 - `npm install mysql2`
- Import the *mysql2* module.
 - Not *mysql*. The *mysql2* version is faster, better protected against SQL injection, natively supports promises (async/await), and is actively developed.
- Create a connection using the `createConnection()` method with connection data.
 - Code in section 1
 - connection is prepared, you can use the `db` variable in other parts of the code.

```
// --- SECTION 1: Configuration & Imports ---
// file: db_ex.js
const express = require('express');
const fs = require('fs');
const path = require('path'); // path.join()
const mysql = require('mysql2');
const app = express();
const port = 3000;

var db = mysql.createConnection({
  host: "localhost",
  user: "admin",
  port: 3306,
  password: "admin12345",
  database: "pwr",
});

app.use(express.urlencoded({ extended: true })); // for POST request
app.use(express.static('public')); // for css.style.css
```

Creating and closing a database connection 2/2

- The web server should only start if the connection is successful.
 - The code is in section 3.
 - Now the `db` variable is the active connection.
- When shutting down the server (e.g., by pressing Ctrl-C in the console, or closing the console), the database connection should be closed. Translating signal handling in *Node.js* is not part of this tutorial, so the code for use without explanation is in section 4.

```
// --- SECTION 3: Startup (The "Engine Launch") ---
db.connect((err) => {
  if (err) {
    console.error("Could not connect to DB!");
    process.exit(1);
  }
  console.log("MySQL connected...");

  // The server starts ONLY when the database confirms the connection
  app.listen(3000, () => {
    console.log("Server is running on port 3000");
  });
});

// --- SECTION 4: Shutdown (The "Cleanup") ---
process.on('SIGINT', () => {
  db.end(() => {
    console.log("DB closed. Exiting...");
    process.exit(0);
  });
});
```

```
C:\Users\dariu\Documents\Dydaktyka\2025-26\semestr zimowy 25_26\WSP-en 25-26\Wykład\W11>node db_ex.js
MySQL connected...
Server is running on port 3000
DB closed. Exiting...
```

```
C:\Users\dariu\Documents\Dydaktyka\2025-26\semestr zimowy 25_26\WSP-en 25-26\Wykład\W11>
```

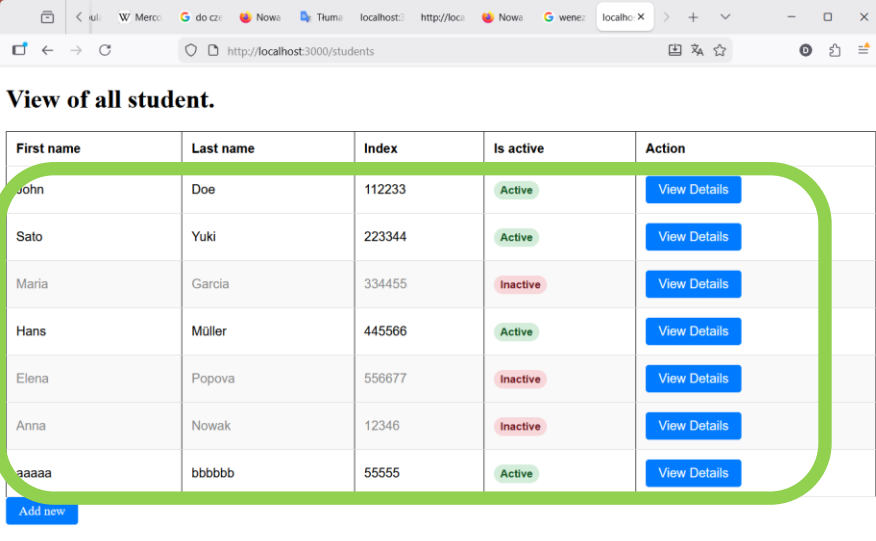
• It is **not recommended** to open and close the connection **for each http request**, although you can write a program this way.

Start of section 2 - rows of students

- The generateStudentsRows() method receives a collection of students (JavaScript objects) with property names from a database table and creates a string containing the HTML code for the table rows
 - along with styling based on the is_active field.
- The code uses lambda expression mapping.

```
let generateStudentsRows=function(db_result){
  return db_result.map(student => {
    // 1. Determine status label and CSS classes based on 'is_active' (0 or 1)
    const statusLabel = student.is_active ? 'Active' : 'Inactive';
    // Class for the status badge (green or red)
    const badgeClass = student.is_active ? 'badge-active' : 'badge-inactive';
    // Class for the entire row (e.g., to make inactive students look "faded")
    const rowClass = student.is_active ? 'row-active' : 'row-dimmed';

    return `
      <tr class="${rowClass}">
        <td>${student.first_name}</td>
        <td>${student.last_name}</td>
        <td>${student.n_index}</td>
        <td>
          <span class="status-badge ${badgeClass}">${statusLabel}</span>
        </td>
        <td>
          <a href="/students/view/${student.id}" class="view-btn">View Details</a>
        </td>
      </tr>
    `;
  }).join('');
}
```



View of all student.

First name	Last name	Index	Is active	Action
John	Doe	112233	Active	View Details
Salo	Yuki	223344	Active	View Details
Maria	Garcia	334455	Inactive	View Details
Hans	Müller	445566	Active	View Details
Elena	Popova	556677	Inactive	View Details
Anna	Nowak	12346	Inactive	View Details
aaaaa	bbbbbb	55555	Active	View Details

[Add new](#)

Handling the GET /students request

- After asynchronously loading the `student_index.html` file, a `SELECT` query is sent to the database using the active `db` connection. The result `result` is received as a collection of student objects with the appropriate properties, such as the column names in the database table.
- After the `result` is parsed from the database into table rows, the resulting text will be replaced in place of the `{allStudentsView}` tag.
- Support for reading the file with an error and communicating with the database with an error is missing; there will be in a further code.

```
<!DOCTYPE html>
<!--File: student_index.html-->
<html>
<head>
<link rel="stylesheet" href="/css/style.css">
</head>
<body>
<h1>View of all student.</h1>
<table>
  <thead>
    <tr>
      <th>First name</th>
      <th>Last name</th>
      <th>Index</th>
      <th>Is active</th>
      <th>Action</th>
    </tr>
  </thead>
  <tbody>
    {allStudentsView}
  </tbody>
</table>
<a href="/students/add" class="view-btn">Add new</a>
</body>
</html>
```

```
app.get('/students', (req, res) => {
  fs.readFile(path.join(__dirname, 'student_index.html'), 'utf-8', (err, htmlTemplate) => {
    db.query("SELECT * FROM students", (err, result) => {
      // Map SQL results to HTML table rows
      const tableRows = generateStudentsRows(result);
      // Now replace the placeholder in your template
      const finalPage = htmlTemplate.replace("{allStudentsView}", tableRows);
      res.send(finalPage);
    });
  });
});
```

Handling GET request `/students/view/:id` - 1/2

- Instead of a fixed URL, the `app.get()` method can have a URL pattern with selected parameters as its first argument.
 - Here, it's the `id` parameter.
- Parameter values are accessed through the object in the `req.params` property:
 - here, `req.params.id`.
- This time, the code handles any errors (file reading and database communication), which ends with a response with a 500 error code and a short error message.

```
app.get('/students/view/:id', (req, res) => {
  const studentId = req.params.id;

  // 1. Read the template file
  fs.readFile('./student_view.html', 'utf8', (err, htmlTemplate) => {
    if (err) {
      return res.status(500).send("Template file missing.");
    }

    // 2. Fetch the specific student from the database
    const sql = "SELECT * FROM students WHERE id = ?";
    db.query(sql, [studentId], (err, result) => {
      if (err) return res.status(500).send("Database error.");

      if (result.length === 0) {
        return res.status(404).send("Student not found.");
      }
    })
  })
})
```

SQL Prepared Statements and template file - `student_view.html`

- It's best to prepare SQL queries in a form that's resistant to SQL injection attacks. Therefore, they're broken down into a query with parameters (?) and a separate array of parameters (in this case, a single-element `[studentId]`), which will be appropriately encoded on the database server side.
 - `const sql = "SELECT * FROM students WHERE id = ?";`
 - `db.query(sql, [studentId], (err, result) => {`
- The template file contains the placeholder `{oneStudentView}`.

```
<!DOCTYPE html>
<!--File: form_table.html-->
<html><head>
<link rel="stylesheet" href="/css/style.css">
</head><body>
<h1>View of selected student.</h1>
<table>
  <tbody>
    {oneStudentView}
  </tbody>
</table>
<a href="/students" class="back-link">&larr; Back to Students List</a>
</body>
</html>
```

Handling GET request /students/view/:id - 2/2

- The result of the SELECT query is a collection, so you should take the element with index 0.
- The rest of the code is similar to the previous URL.
- Theoretically, instead of using a parameter in the URL path, you could use a parameter in the query string : /students/view?id=X
 - then you should use req.query.id.

```
const student = result[0];

// 3. Prepare the HTML snippet for the placeholder
const statusLabel = student.is_active ? "Active" : "Inactive";
const badgeClass = student.is_active ? "badge-active" : "badge-inactive";

const detailHtml = `
  <div class="profile-card">
    <h2>${student.first_name} ${student.last_name}</h2>
    <p><strong>Database ID:</strong> ${student.id}</p>
    <p><strong>Index Number:</strong> ${student.n_index}</p>
    <p><strong>Status:</strong>
      <span class="status-badge ${badgeClass}">${statusLabel}</span>
    </p>
  </div>
`;

// 4. Replace placeholder and send response
const finalPage = htmlTemplate.replace("{oneStudentView}", detailHtml);
res.send(finalPage);
  });
});
```

Handling GET request /students/add

- To po prostu wysłanie zawartości pliku `student_add.html` ze wzorcem formularza.

```
app.get('/students/add', (req, res) => {  
  res.send(fs.readFileSync('student_add.html', 'utf8'));  
});
```

```
<!DOCTYPE html>  
<!-- file: student_add.html -->  
<html><head>  
</head><body>  
<h1> Add a new student </h1>  
<p> Fullfill the form for a new student. </p>  
<form action="http://localhost:3000/students/add" method="POST">  
  <label for="fname">First name:</label><br>  
  <input type="text" id="fname" name="fname" placeholder="Anna" required><br>  
  <label for="lname">Last name:</label><br>  
  <input type="text" id="lname" name="lname" placeholder="Nowak" required><br><br>  
  <label for="index_no">Student ID:</label><br>  
  <input type="number" id="index_no" name="index_no" placeholder="00000" required><br><br>  
  <label for="is_active">Is active?</label><br><br>  
  <input type="radio" id="is_active" name="is_active" value="1" checked>YES<br>  
  <input type="radio" id="is_active" name="is_active" value="0">NO<br>  
  <input type="submit" value="Submit">  
</form>  
</body>  
</html>
```

Add a new student

Fullfill the form for a new student.

First name:

Last name:

Student ID:

Is active?

☒ YES☐ NO

Handling POST request `/students/add`

- Retrieving data from the form using `req.body`.
- Processing it into the data types found in the database table.
- Executing the INSERT command, handling any errors.
- Sending back a response containing information about the need to redirect to the GET `/students` request..
 - `res.redirect('/students');`

```
app.post('/students/add', (req, res) => {
  const firstNameFromForm = req.body.fname; // names from form
  const lastNameFromForm = req.body.lname;
  const indexNumberFromForm = req.body.index_no;
  // Radio buttons send the 'value' as a string ("1" or "0")
  const isActiveStatus = parseInt(req.body.is_active);
  // 2. Prepare SQL with DATABASE column names
  const sql = "INSERT INTO students (first_name, last_name, n_index, is_active) VALUES (?, ?, ?, ?)";
  // 3. Map form variables to SQL placeholders
  const values = [firstNameFromForm, lastNameFromForm, indexNumberFromForm, isActiveStatus];

  db.query(sql, values, (err, result) => {
    if (err) {
      console.error("Database Error:", err);
      return res.status(500).send("Could not add student to database.");
    }
    console.log(`Success! New student added with DB ID: ${result.insertId}`);
    // 4. Redirect back to the main list
    res.redirect('/students');
  });
});
```

Summary of requests and related mechanisms

Method	Path	Aim	New Mechanisms
GET	/students	Displaying a list of all students	Data mapping (<code>.map()</code>), Placeholder in HTML (<code>{studentsTable}</code>), Conditional Rendering (CSS classes for status)
GET	/students/view/:id	View details of one student	Parameters in URL (<code>:id</code>), <code>req.params</code> , SQL Prepared Statements (?), Error handling (404 when no ID)
GET	/students/add	Displaying the form for adding a new student	Not new Mechanisms: Serving static HTML files, Form structure (name, method, action)
POST	/students/add	Saving a new student to the database	Middleware (<code>urlencoded</code>), Handling <code>req.body</code> , Field name mapping (<code>fname</code> to <code>first_name</code>), Redirection (<code>res.redirect</code>)